CoProMem: Structural Procedural Memory for Agent Workflows

Anonymous Author(s)
Submission Id: TBD

Abstract

Agent memory allows language agents to reuse experience across tasks, but useful reuse requires more than retrieving a related past case. Existing approaches encode experience as reflections [12], distilled strategies [1, 8], or workflows [17]; these memories can leave operation dependencies implicit, and relevance-based retrieval does not establish whether a procedure fits the new task's inputs and constraints. Repeated memory synthesis and retrieval add computational cost [2, 23], while inappropriate reuse can propagate errors [19]. We introduce CoProMem, which extracts an ordered operation core common to successful execution graphs and retains input–output slot relations only when observed across those graphs. For a new task, it checks the public task query against the learned terminal operation and supported slot relations, withholding guidance when compatibility is ambiguous or unsupported. This makes procedural applicability explicit without embedding-based ranking or repeated memory rewriting. Experiments on AppWorld and ALFWorld demonstrate competitive task performance against established agent-memory baselines, with CoProMem achieving the highest reported task-success metrics across both benchmarks and the lowest average cost on AppWorld.

Keywords

agent memory, procedural memory, structural retrieval, language agents

ACM Reference Format:
Anonymous Author(s). 2027. CoProMem: Structural Procedural Memory for Agent Workflows. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027*, IFAAMAS, 6 pages.

1 Introduction

Tool-using agents encounter tasks that share operations while differing in goals and input values. Reusing experience requires two decisions: what procedure to retain from past executions, and whether that procedure fits a later task.

A successful execution may include exploration, retries, and task-specific values. A valid tool call can also occur in a failed task. We therefore distinguish whole-task success from call-level validity when extracting an ordered operation core and the input–output slot relations shared by successful executions.

An earlier task's relevance does not establish a procedure's applicability. An agent that searched for and followed an artist may later be asked only to inspect that artist; even another follow request needs a current identifier. CoProMem therefore checks the new instruction and callable interface before presenting guidance. Abstention is a defined outcome when public support is insufficient, consistent with documented risks of misleading experience reuse [19].

CoProMem stores validated schemas containing the common operation core and attested slot relations, then tests their structural compatibility with a later task's public description. This separates construction from selection without changing the agent model's parameters. Unlike memories expressed as verbal feedback or insights [12, 24], CoProMem represents callable operations and their observed slot relations explicitly; related workflow and procedural-memory methods are discussed in Section 2.

Contributions. We define a schema built from an operation sequence common to successful trials and attested input–output slot relations, together with a public-task retrieval rule that can abstain. In the reported comparisons, CoProMem leads on AppWorld Avg@3, Pass@3, and average cost, and on ALFWorld completion. An AppWorld exposure analysis describes outcomes with and without retrieved guidance.

2 Related Work

Context management and current state. Agents can preserve information outside a model call while controlling what enters its limited context. MemGPT [9] moves information across memory tiers, and MemoryOS [4] organizes conversational memory by duration. COMPASS [16] and U-Fold [14] construct compact views of ongoing interactions. These systems manage context or conversational state; CoProMem stores procedures extracted after scored tasks.

Persistent episodic and structured memory. Generative Agents [10] records experiences and synthesizes reflections for planning; MemoryBank [25] maintains conversational memory.

A-Mem [20], MAGMA [3], and SEEM [5] organize retained information for retrieval. CoProMem stores an ordered tool procedure with public input–output slot relations and tests its applicability to a new task's exposed operations and inputs.

Experience as reusable action guidance. Reflexion [12] stores verbal reflections on feedback, while ExpeL [24] extracts natural-language insights from experience. Agent Workflow Memory [17] induces reusable action routines and selectively provides them for later tasks. ReasoningBank [8] distills strategies from successful and failed experiences for retrieval; ReMe [1] distills, reuses, and refines procedural lessons. CoProMem stores an ordered operation core and attested input–output slot relations, then checks the terminal operation and required inputs against public task information.

Writing, revising, and withholding memory. Memory construction and retrieval also require control policies. Nemori [6] selects experience for distillation, while RecMem [2] delays extraction until related interactions recur. Memory-R1 [21] learns memory-management decisions. These policies differ from CoProMem's

This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

deterministic task-boundary update and public-task compatibility test. Empirical evidence of error propagation and misleading experience replay [19] motivates care in deciding when to present stored guidance.

Memory evaluation. LoCoMo [7] and LongMemEval [18] assess memory over extended conversations; Mem2ActBench [11] asks whether retained information supports later tool use. Our task comparisons use AppWorld's interactive API environment [15] and ALFWorld's text-based household tasks [13].

3 Problem Formulation

3.1 Sequential Tool-Use Setting

We consider an ordered sequence of tasks indexed by t. Task t provides a public instruction x_t and public domain and application descriptions $\mathcal{P}_t$. A frozen registry R specifies callable operations and their public input and output slots. The agent may perform n_t trials of the task, each in a fresh environment; this configured count determines the schema-admission threshold in Section 4.4. Trial i produces an ordered sequence of tool-call attempts $C_{t,i}$ and receives a score $s_{t,i} \in [0,1]$ and a full-success label $y_{t,i} = \mathbf{1}[s_{t,i} = 1]$ from the scorer.

Let $E_{t,i} = \mathrm{ValidCalls}_R(C_{t,i})$ retain calls with arguments accepted by the tool schema and a successful tool response. This call-level filter and $y_{t,i}$ express different evidence: a response-successful call can occur in a failed trial, and a successful trial can contain unsuccessful attempts. The scored evidence available after task t is

$$B_t = \big((y_{t,i}, E_{t,i})\big)_{i=1}^{n_t}. \tag{1}$$

At an interaction turn k, the model-visible context $c_{t,i,k}$ contains the current instruction, preceding messages and tool responses, and public tool specifications. The environment condition $q_{t,i,k}$, including task facts and progress, is trial-local and may be only partly exposed through those responses.

3.2 Procedural Abstraction

The first problem is to extract a reusable procedure from a scored task batch. A successful trace contains concrete inputs and may include incidental calls; response-successful calls alone do not identify a successful task. The abstraction should retain an ordered core of public operations and the output slots that match later required input slots. In the artist example, a search output can match the identifier supplied to a follow call, without retaining the earlier artist name or identifier.

Successful trials in B_t provide the operation sequence and shared slot relations for a candidate schema σ_t. Equality of recorded field values attests a slot relation; it does not prove that the later call causally depended on the earlier response. Section 4.4 specifies the construction and validation rules.

3.3 Procedural Applicability

The second problem is whether a stored schema should guide a new task. Let M_t be the external bank of validated schemas available before task t. Selection may use the public instruction x_t, descriptions $\mathcal{P}_t$, and registry R, but not calls, responses, scores, or hidden environment facts from task t. Their pre-execution descriptor is $d_t = \mathrm{Query}(x_t, \mathcal{P}_t; R)$.

For a candidate schema, the decision should ask whether the public task supports its intended terminal operation and the inputs required by its exposed prerequisites. It must also allow no selection when support is absent or multiple candidates remain equally supported. We write $(g_t, \rho_t) = \mathrm{Retrieve}(M_t, d_t; R)$, where g_t is model-visible guidance, ρ_t records the decision, and $g_t = \epsilon$ means abstention from learned guidance. When nonempty, g_t enters the model context; abstention does not stop the agent from acting. Structural compatibility based on public information is a condition for presenting a procedure, not proof that it will satisfy constraints hidden in the new environment.

3.4 Task-Boundary Update and Evaluation

All trials of task t receive the same g_t retrieved from the same pre-task bank. Only after every trial has been scored can the bank change:

$$M_{t+1} = \mathrm{Update}(M_t, B_t; R). \tag{2}$$

This boundary prevents a trial from using a schema induced from another trial of the same task. The bank is external state, distinct from model parameters, model-visible context, and trial-local environment state; CoProMem does not optimize the model parameters. For any set $\mathcal{I}$ of scored task–trial pairs, the whole-task success rate is $\bar{y}_{\mathcal{I}} = |\mathcal{I}|^{-1} \sum_{(t,i)\in\mathcal{I}} y_{t,i}$. Other task-level measures are specified with the evaluation protocol in Section 5.

4 Method

4.1 Overview

CoProMem separates schema construction from retrieval (Figure 1). Before task t, retrieval tests schemas in M_t against its public descriptor d_t. After the task is scored, successful executions in B_t provide a candidate schema for the next bank state. The agent system supplies the model, tools, environment, and scorer; CoProMem supplies guidance g_t and updates the bank after scoring.

4.2 Public Interface and Schema Representation

The frozen registry R defines canonical operations, access modes, public input and output slots, optional declared dependencies, and aliases. Its identity is bound to each schema; a schema from a different registry is ineligible for retrieval. The registry describes the callable interface rather than an observation from a trial.

A stored schema σ consists of an ordered sequence of distinct operation occurrences $V_\sigma = (v_1, \ldots, v_m)$, attested slot-level relations D_σ, and a terminal occurrence or consecutive terminal group T_σ. Each occurrence identifies an operation, access mode, and public required input and output slots. The schema also carries the registry identity and evidence identities for validation. Separate occurrence positions represent repeated calls; concrete argument and response values are absent.

4.3 Constructing Evidence from Scored Trials

Each trial's response-successful calls $E_{t,i}$ form an ordered evidence graph. CoProMem projects it onto domain operations while recording excluded discovery, authentication, and supervisor calls for

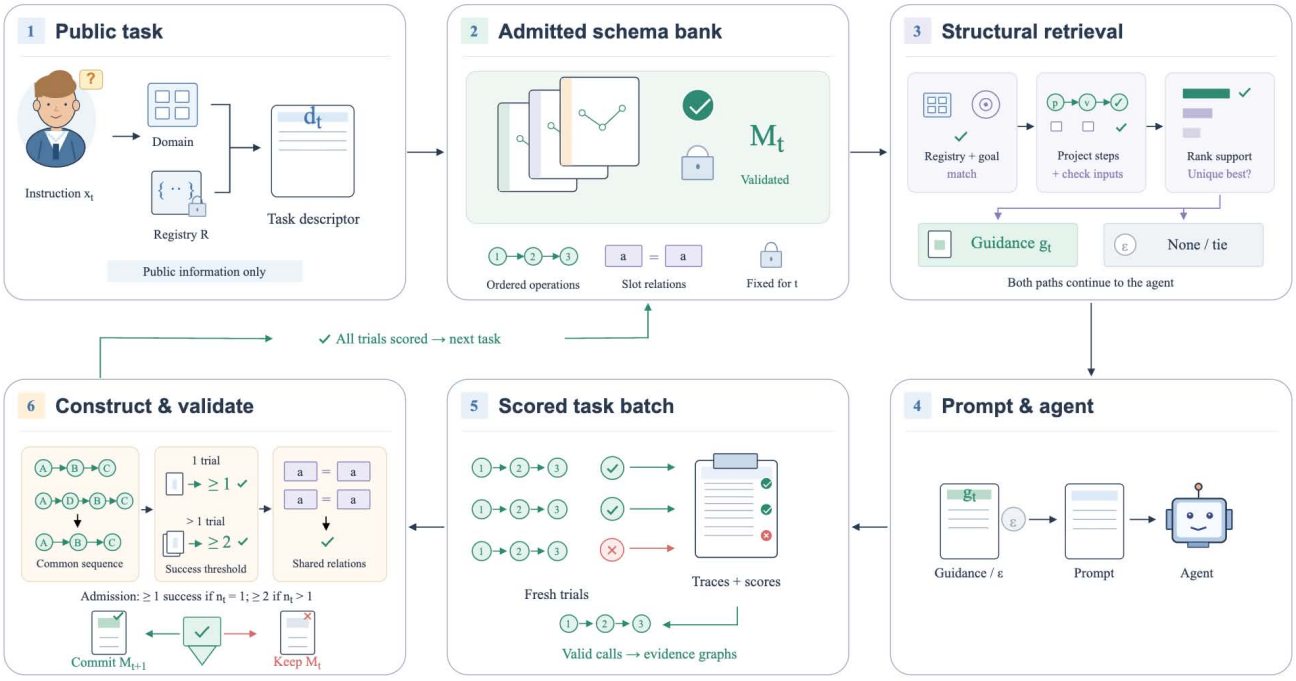


Figure 1: CoProMem's task-level workflow. Public-task retrieval uses the pre-task bank; after scoring, successful executions provide the operation core and attested slot relations for the next bank update.

audit. Let $G_{t,i}$ be the projected graph and $D_{t,i}$ its observed slot relations. Each retained occurrence records an operation, access mode, and public slots. In the artist example, the search response and later follow input carry matching identifiers. For occurrences $j < \ell$, a relation from output slot u to input slot v is recorded exactly when their value digests agree:

$$(j,u) \to_{G_{t,i}} (\ell, v) \quad \Longleftrightarrow \quad j < \ell \ \wedge \ h\big(\mathrm{out}_j[u]\big) = h(\mathrm{in}_\ell[v])\,. \tag{3}$$

Here j and ℓ index retained calls and h is a deterministic digest. Call order and registry-declared dependencies are also recorded, but neither alone establishes an observed slot relation. Equal digests attest field-value equality, not a causal dependence of the later call on the earlier response. The schema retains public slots and occurrence positions, not values or digests. Repeated operations remain distinct; the whole-task label $y_{t,i}$ determines whether a graph contributes to schema construction.

4.4 Inducing and Validating a Schema

Let $\mathcal{J}_t^+ = \{i : y_{t,i} = 1\}$ index the successful trials. When this set is nonempty, CoProMem scans the first successful graph and greedily matches each operation–access-mode occurrence in order across the other successful graphs. The retained sequence L_t is common to those graphs; the scan does not compute a longest common subsequence. The operation core is

$$V_t = L_t, \qquad V_{\sigma_t} = V_t. \tag{4}$$

Repeated occurrences retain distinct positions. The last retained write operation is the preferred terminal; if none remains, the final retained domain operation is used.

The operation core alone does not state which outputs can supply later inputs. CoProMem aligns its retained occurrences to each successful graph and keeps only slot relations observed at those aligned positions in every success. Let $\alpha_i(D_{t,i})$ express graph i's observed relations in the aligned positions. Then

$$D_{\sigma_t} = \bigcap_{i \in \mathcal{J}_t^+} \alpha_i(D_{t,i}). \tag{5}$$

The artist search-to-follow relation is therefore retained only if the aligned slots match in every successful graph, even when the concrete identifiers differ. Alignment remains occurrence-specific when operations repeat. The intersection attests repeated equality, not causal flow.

Validation prevents an unsupported or malformed core from entering the bank. The minimum number of successful trials depends on the configured trial count:

$$m(n_t) = \begin{cases} 1 & \text{if } n_t = 1, \\ 2 & \text{if } n_t > 1. \end{cases} \tag{6}$$

A candidate meets the successful-trial requirement only if $|\mathcal{J}_t^+| \geq m(n_t)$. It must also have a nonempty domain core, a valid terminal, distinct occurrence identities, and forward slot relations between declared public slots. Repeated terminal occurrences form a terminal group only when they are a supported consecutive repetition.

The candidate, evidence, and decision have deterministic identities. For proposed schema σ_t, the update is

$$M_{t+1} = \begin{cases} \text{Commit}(M_t, \sigma_t), & \text{if } \sigma_t \text{ passes validation,} \\ M_t, & \text{otherwise.} \end{cases} \tag{7}$$

An identical content-derived schema is not duplicated. The update leaves the agent model and its parameters unchanged.

4.5 Retrieval from a Public Task Description

To address procedural applicability, CoProMem evaluates stored schemas using only d_t, which is derived before execution from x_t, $\mathcal{P}_t$, and R. A fixed lexical rule supports an operation when the instruction matches its action and a callable noun; an explicitly named application can support a callable with no specific noun. Let Q_t be the supported operations. This lexical evidence can miss paraphrases, and it cannot inspect hidden state or future execution.

Only the portion connected to a schema's terminal operation is considered for guidance. For schema σ, projection P_σ follows attested slot relations backwards from terminal occurrence or group T_σ to include prerequisites and forwards to include reachable verification steps. Its occurrence set is

$$V(P_\sigma) = \text{Anc}_{D_\sigma}(T_\sigma) \cup \text{Desc}_{D_\sigma}(T_\sigma), \tag{8}$$

Both closures include T_σ. Call order and declared registry dependencies cannot add steps to this projection.

The compatibility test checks the terminal operation and each projected required input. The terminal must belong to Q_t. An input is supported either by an incoming attested slot relation, as with the follow call's artist identifier, or by public query support for both the operation and its external input. Let $D(P_\sigma)$ be the projected attested relations, $K(u)$ the non-generic lexical concepts of input slot u, and $K(Q_t)$ the concepts of supported operation names. For each required input of occurrence v, the rule is

$$\begin{aligned} \text{InputOK}_t(v,u) \iff & \left[\exists (w,p)\colon\ (w,p) \to (v,u) \in D(P_\sigma)\right] \\ & \vee \left[\text{op}(v) \in Q_t \ \wedge\ \varnothing \neq K(u) \subseteq K(Q_t)\right]. \end{aligned} \tag{9}$$

An operation from a different application additionally requires that application to be publicly named as relevant. The rule tests public support for the stored structure; it does not check whether future argument values or hidden conditions will be correct.

Several schemas can pass compatibility. CoProMem ranks them by how many distinct nonterminal operations in the projected procedure also have public query support, without using historical success count as a tie-breaker. Let C_t contain compatible schemas bound to R. For $\sigma \in C_t$, the score is

$$r_t(\sigma) = \left|\{\text{op}(v) : v \in V(P_\sigma),\ \text{op}(v) \neq \text{op}(T_\sigma),\ \text{op}(v) \in Q_t\}\right|. \tag{10}$$

Here $\text{op}(T_\sigma)$ denotes the shared operation of a terminal group or its sole occurrence. If exactly 1 compatible schema maximizes r_t, its projected operations and public slots become g_t; with no compatible schema or a tie, $g_t = \epsilon$. Guidance asks the agent to verify current-task values for inputs without an incoming slot relation. Consecutive identical occurrences may be described as a repetition while retaining their separate identities in provenance ρ_t. That record binds the public query, bank and registry identities, candidate decisions, selected schema, and guidance. Retrieval can be reproduced from these inputs and leaves M_t unchanged.

4.6 Agent Integration and Computational Scope

Retrieval occurs once before the task's trials. In each fresh trial, nonempty g_t enters the initial model-visible prompt alongside x_t, and public tool specifications accompany model calls. Validated tool requests produce responses for the next context $c_{t,i,k}$, following the action–observation setting of ReAct [22]. After a final answer or interaction limit, the scorer supplies $s_{t,i}$ and $y_{t,i}$; invalid and unsuccessful attempts remain in the trace but do not enter $E_{t,i}$. The complete scored batch is required before the update in Eq. 7.

Retrieval scans committed schemas without a learned embedding index or eviction rule. CoProMem applies no separate context-compression stage. The consequences of bank growth are discussed in Section 8.

5 Experimental Setup

We evaluate CoProMem on the validation sets of AppWorld [15], an interactive API benchmark, and ALFWorld [13], a household-task benchmark with 134 evaluated tasks. The comparison includes no memory, ReMe [1], and ReasoningBank [8]. ReMe and ReasoningBank were cloned from their source repositories; the ReMe arm uses its dynamic memory-update variant.

All experiments use DeepSeek-V4.1-Flash through the OpenRouter API. Each arm starts with an empty memory bank and uses the same base prompts. Tasks follow each dataset's default order. CoProMem applies Eq. 6 after each scored task batch: admission requires two successes in AppWorld's three-trial setting and one in ALFWorld's single-trial setting.

AppWorld reports Avg@3, the mean of the binary full-success labels $y_{t,i}$ across three trials per task, averaged over all evaluated tasks. Pass@3 is the number of tasks with at least one successful trial divided by the number of evaluated tasks. We also report mean actions and USD cost. ALFWorld uses one trial per task and reports task completion, mean steps, and reported USD cost. The retrieval-exposure analysis compares CoProMem with no memory on 168 AppWorld tasks; its exposure groups are observational.

6 Results

Tables 1 and 2 report task outcomes and resource use.

On AppWorld, CoProMem records the highest Avg@3 (80.95%) and Pass@3 (83.33%) among the four arms. ReMe is next on both measures, followed by ReasoningBank and no memory. CoProMem also has the lowest reported average cost, while ReMe uses the fewest actions.

Method	Avg@3	Pass@3	Actions	Cost
No memory	71.24	72.62	18.46	<u>0.77</u>
ReMe	<u>77.38</u>	<u>78.57</u>	**17.85**	1.34
ReasoningBank	73.81	74.41	20.48	0.82
CoProMem	**80.95**	**83.33**	<u>18.15</u>	**0.75**

Table 1: AppWorld task outcomes (scores in %; mean actions and USD cost).

On ALFWorld, CoProMem achieves the highest single-pass completion rate (88.06%), with ReasoningBank next. CoProMem uses fewer steps and incurs lower reported cost than no memory and ReMe; ReasoningBank is the most efficient arm on both measures.

Thus, CoProMem ranks first or second on each reported metric across the two benchmarks.

Method	Success (%)	Avg. steps	Avg. Cost (USD)
No memory	79.11	23.12	1.2700
ReMe	82.09	25.09	1.3334
ReasoningBank	85.08	**19.06**	**0.9056**
CoProMem	**88.06**	20.91	1.1722

Table 2: ALFWorld task outcomes. Success is task completion.

Retrieval exposure. The AppWorld comparison with no memory covers 168 tasks: CoProMem supplies guidance on 102 and abstains on 66. Table 3 reports Avg@3 on a 0–1 scale; Table 1 reports percentages.

Using CoProMem (C) and no memory (N), the task-level Avg@3 contrast is written as

$$\Delta_t = \text{Avg@3}_{\text{C},t} - \text{Avg@3}_{\text{N},t}\,. \tag{11}$$

Across all tasks, CoProMem exceeds no memory by 0.098 in Avg@3. The reported differences are 0.138 for guidance-exposed tasks and 0.036 for abstained tasks. Retrieval determines exposure, so these subgroup comparisons describe the observed policy rather than the effect of guidance.

Comparison	Tasks	Avg@3 C/N	Difference
All	168	0.810/0.712	+0.098
Guidance	102	0.837/0.699	+0.138
Abstained	66	0.768/0.732	+0.036

Table 3: AppWorld retrieval exposure.

7 Discussion

7.1 Structural Memory and Observed Performance

CoProMem's schema records an operation core shared by successful executions and slot relations attested across them. A separate public-task test checks the terminal operation and required inputs before presenting that schema as guidance. The test may abstain when support is insufficient or the best-ranked schemas tie.

The benchmark comparisons show CoProMem's strongest task outcomes on both validation sets, with the lowest average cost on AppWorld. They also show a distinct efficiency pattern: ReMe uses fewer AppWorld actions, while ReasoningBank uses fewer ALFWorld steps at lower cost. The exposure analysis records when guidance was supplied and the outcomes in each group; it does not separate retrieval's contribution from task selection.

7.2 Design Implications

Observed slot equality and public-task compatibility make the basis for each retrieved procedure inspectable without asserting causal dependence between tool calls. The greedy first-success anchor, lexical matching under paraphrase, and bank growth are natural targets for future study. Controlled retrieval interventions would be needed to attribute outcome differences to particular design choices.

8 Limitations

Resource constraints limited evaluation to the AppWorld and ALFWorld validation sets; WebArena and the benchmarks' test sets were not evaluated. ALFWorld used one trial per task because it was comparatively easy for DeepSeek-V4.1-Flash in this setting. Completion rates and method rankings may differ under three trials or other model settings.

The comparisons evaluate CoProMem as a complete system; a comprehensive ablation study was not conducted. Sensitivity of the greedy common sequence to its first successful graph has not been measured.

9 Conclusion

CoProMem builds procedural schemas from operation sequences and slot relations shared across successful executions, then checks their compatibility with a new task's public description. In the reported validation-set comparisons, it achieves the highest AppWorld Avg@3 and Pass@3, the lowest AppWorld average cost, and the highest ALFWorld completion rate. Its explicit construction, task-boundary update, and abstaining retrieval rule provide a basis for studying procedural reuse across tasks.

References

[1] Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. 2026. Remember Me, Refine Me: A Dynamic Procedural Memory Framework for Experience-Driven Agent Evolution. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics. doi:10.18653/v1/2026.findings-acl.829

[2] Zijie Dai, Shiyuan Deng, Sheng Guan, Yizhou Tian, Xin Yao, Xiao Yan, and James Cheng. 2026. RecMem: Recurrence-based Memory Consolidation for Efficient and Effective Long-Running LLM Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics. doi:10.18653/v1/2026.findings-acl.1619

[3] Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. 2026. MAGMA: A Multi-Graph based Agentic Memory Architecture for AI Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.1709

[4] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory OS of AI Agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. doi:10.18653/v1/2025.emnlp-main.1318

[5] Zhengxuan Lu, Dongfang Li, Yukun Shi, Beilun Wang, Longyue Wang, and Baotian Hu. 2026. Structured Episodic Event Memory. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.277

[6] Wenquan Ma, Jiayan Nan, and WenLong Wu. 2026. What Deserves Memory: Adaptive Memory Distillation for LLM Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.1607

[7] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2024.acl-long.747

[8] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. In *International Conference on Learning Representations*. arXiv:2509.25140 https://openreview.net/forum?id=jL7fwchScm

[9] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv preprint. arXiv:2310.08560 https://arxiv.org/abs/2310.08560

[10] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. doi:10.1145/3586183.3606763

[11] Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. 2026. Mem2ActBench: A Benchmark for Evaluating Long-Term Memory Utilization in Task-Oriented Autonomous Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.370

[12] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*. doi:10.52202/075280-0377

[13] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *International Conference on Learning Representations*. https://openreview.net/forum?id=0IOX0YcCdTn

[14] Jin Su, Runnan Fang, Yeqiu Li, Xiaobin Wang, Shihao Cai, Pengjun Xie, Ningyu Zhang, and Fajie Yuan. 2026. U-Fold: Dynamic Intent-Aware Context Folding for User-Centric Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics. doi:10.18653/v1/2026.findings-acl.897

[15] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 16022–16076. doi:10.18653/v1/2024.acl-long.850

[16] Guangya Wan, Mingyang Ling, Xiaoqi Ren, Rujun Han, Sheng Li, and Zizhao Zhang. 2026. COMPASS: Enhancing Agent Long-Horizon Reasoning with Evolving Context. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.152

[17] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning*, Vol. 267. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[18] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html

[19] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. 2026. How Memory Management Impacts LLM Agents: An Empirical Study of Experience-Following Behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.27

[20] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems*. doi:10.52202/085713-0593

[21] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schuetze, Volker Tresp, and Yunpu Ma. 2026. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.583

[22] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. arXiv:2210.03629 https://arxiv.org/abs/2210.03629

[23] Jiaquan Zhang, Chaoning Zhang, Shuxu Chen, Zhenzhen Huang, Pengcheng Zheng, Zhicheng Wang, Ping Guo, Fan Mo, Sung-Ho Bae, Jie Zou, Jiwei Wei, and Yang Yang. 2026. Lightweight LLM Agent Memory with Small Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics. doi:10.18653/v1/2026.acl-long.588

[24] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*. doi:10.1609/aaai.v38i17.29936

[25] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*. doi:10.1609/aaai.v38i17.29946